

Final Project: Physics-Informed Adaptive Traffic Control

Course: Machine Learning (2025 Fall)

Date: 15/05/2025

Author: Alvin B. Lin

Student ID: 112652040

Problem 1: Future AI Capability (20-Year Horizon)

1. The Capability: City-Scale Autonomous Flow Orchestration

Currently, traffic control systems are fragmented. Traffic lights operate on fixed cycles or simple local sensors, unable to communicate with neighboring intersections effectively.

In 20 years, I envision an AI system capable of **“City-Scale Autonomous Flow Orchestration.”** This system will not merely “react” to traffic; it will treat the entire city’s vehicle fleet as a continuous fluid dynamic system. It will coordinate thousands of intersections simultaneously in real-time to eliminate “phantom jams” and the “ghost green light” phenomenon (where a green light serves an empty road while cross-traffic waits).

2. Why It Matters?

Traffic congestion is not just a nuisance; it is a massive economic drain and a primary contributor to urban carbon emissions. A system that can mathematically optimize flow efficiency by even 20% would significantly reduce fuel consumption, lower anxiety for millions of commuters, and transform urban logistics. The ability to dynamically re-route flow during emergencies (e.g., ambulances, natural disasters) would also save lives.

Problem 2: Ingredients and Resources

To realize this vision, the following components are essential:

- **Data (Multimodal & Real-Time):** The system requires more than just car counts. It needs real-time *density maps* generated from fused data sources: government surveillance cameras, GPS trajectories from private vehicles (Google Maps API), and inductive loop sensors. The data must be high-resolution (per-lane) and low-latency.
- **Tools (Physics + Computation):**
 - **PDEs (Partial Differential Equations):** To model the conservation of vehicles (Lighthill-Whitham-Richards model) so the AI understands that cars cannot teleport.
 - **Graph Neural Networks (GNNs):** To model the spatial dependency between intersections (nodes) and roads (edges), allowing the system to learn how a red light at intersection A affects intersection B two miles away.
- **Hardware & Environment:** Implementation requires **Edge Computing** nodes at every major intersection to process video feeds locally (reducing latency) and a centralized cloud cluster for global policy updates.

- **Learning Setup:** A **Multi-Agent Reinforcement Learning (MARL)** framework is needed, likely initialized via *Imitation Learning* (mimicking optimal control theory) and refined via *Online Learning* to adapt to daily variance.

Problem 3: Machine Learning Paradigm

The realization of this capability primarily relies on **Reinforcement Learning (RL)**.

Why Do We Use RL?

Traffic control is a sequential decision-making problem where the “correct” action (switching a light) depends on the future state of the system. There are no static labels (like in Supervised Learning) for what the “perfect” light timing is; there is only a long-term objective (minimize total wait time).

- **State (s_t):** The current queue lengths and moving densities on all road segments.
- **Action (a_t):** The phase selection (Green/Red) for every signal in the network.
- **Reward (r_t):** The negative total waiting time (penalty for congestion).
- **Environment Interaction:** The AI’s decision directly alters the traffic flow, creating a feedback loop that only RL can optimize effectively.

Problem 4: Solvable Model Problem (The First Step)

As a first step toward this 20-year goal, we designed and solved a simplified “Toy Model”: **Adaptive Control on a Barcelona-style Grid.**

1. Problem Design

- **Input:** Queue lengths at each intersection.
- **Output:** Binary signal phase (0: North-South Green, 1: East-West Green).
- **Physics:** We use the **Cell Transmission Model (CTM)** to simulate realistic traffic shockwaves, ensuring cars obey conservation laws.

2. Model and Method

We implemented a **Max Pressure** agent. While not a deep neural network, Max Pressure is a greedy policy derived from queuing theory that acts as an optimal “proxy” for RL. It calculates the pressure P of a phase as the difference between upstream demand and downstream capacity:

$$P_{phase} = \sum(\text{Queue}_{in}) - \sum(\text{Queue}_{out})$$

The agent selects the phase with the highest pressure, ensuring traffic is pushed into empty roads rather than jammed ones.

3. **Implementation and Results** We implemented the simulation in Python (code excerpt in Appendix). We conducted a sensitivity analysis by varying the grid size from 4×4 to 20×20 under heavy traffic inflow.

Table 1: Experiment Results: Fixed Time vs. Smart Agent

Grid Size	Fixed Avg Wait	Agent Avg Wait	Improvement
4×4	425.6	404.5	4.97%
6×6	1353.4	1101.6	18.60%
20×20	17229.0	15173.6	11.93%

Key Observation: In small grids (4×4), the improvement is marginal ($\approx 5\%$), as fixed timing is sufficient. However, as complexity grows ($N \geq 6$), the Smart Agent consistently reduces waiting times by 12 – 18%. In the 20×20 case, this translates to saving over 2,000 vehicle-hours of delay.

4. Discussion

This simplified model taught us that **scalability is the hardest challenge**. A strategy that works for a single intersection often fails in a grid due to “spillback” (jams blocking upstream nodes).

The success of the Max Pressure agent reveals that the key to the future 20-year goal lies in **decentralized decisions with local awareness**. Instead of a single super-computer controlling every light (which is brittle), future AI should likely be composed of thousands of local agents cooperating via graph connections, just as our Max Pressure agents solved the gridlock by managing local pressure gradients.

A Appendix: Core Simulation Code (Python)

Below is the core logic for the Intersection agent and the TianGrid physics update used in our experiment.

```
1 class Intersection:
2     def get_pressure(self):
3         # Calculate Pressure = Upstream Queue - Downstream Queue
4         p_ns = 0; p_ew = 0
5
6         # North-South Pressure
7         if 'N' in self.in_roads:
8             p_ns += self.in_roads['N'].get_stopped_queue()
9             # Subtract downstream congestion to prevent blocking box
10            if 'S' in self.out_roads:
11                p_ns -= self.out_roads['S'].get_total_cars() * 0.5
12
13            # East-West Pressure
14            if 'E' in self.in_roads:
15                p_ew += self.in_roads['E'].get_stopped_queue()
16                if 'W' in self.out_roads:
17                    p_ew -= self.out_roads['W'].get_total_cars() * 0.5
18
19            return max(0.1, p_ns), max(0.1, p_ew)
20
21    def step_flow(self):
22        # The Action: Switch Phase based on Pressure
23        if self.phase_timer >= MIN_GREEN:
24            p_ns, p_ew = self.get_pressure()
25            if self.phase == 0 and p_ew > p_ns:
26                self.switch_phase(1)
27            elif self.phase == 1 and p_ns > p_ew:
28                self.switch_phase(0)
29
30 class DirectedRoad:
31     def step_physics(self, flow_in, flow_out):
32         # CTM: Update density based on conservation law
33         self.cells[-1] = max(0, self.cells[-1] - flow_out)
34         self.cells[0] += flow_in
35
36         # Move cars forward if space allows (Supply/Demand check)
37         for i in range(len(self.cells) - 2, -1, -1):
38             flux = min(self.cells[i], MAX_CAPACITY - self.cells[i+1])
39             self.cells[i] -= flux
40             self.cells[i+1] += flux
```